

MODULE – I

DATABASE SYSTEMS

Dr. Geetha Mary A
Professor ,
SCOPE, VIT, Vellore

Sources:

Pearson Education, Inc. 2011, Elmasri/Navathe, Fundamentals of Database Systems, Sixth Edition
McGraw Hill Education , 2010, Silberschatz, Korth and Sudarshan, Database System Concepts, Sixth edition

- Why to store data?
- Data : Known facts that can be recorded and have an implicit meaning
- Database: Collection of related data
- DBMS: Data Base Management System
 - A software package/ system to facilitate the creation and maintenance of a computerized database.
- Database Systems:
 - The DBMS software together with the data itself.

Some of the DBMS software

- Oracle
- Ms-Access
- Foxpro
- dbase

Database System Application

- Banking
- Airlines
- Universities

Need

- Need WRT **Paper work** (Hard copy of data)
 - ▣ Paper gets degraded over time
- Need WRT **File system**
 - ▣ Store the data in OS file types
 - Eg: Notepad (*.txt)
MS word (*.doc)

File System Contrn...

- To allow users to manipulate information the system application programs that manipulate the files, including
 - ▣ A program to debit or credit an account
 - ▣ A pgm to add a new account
 - ▣ A pgm to find the balance of an account
 - ▣ A pgm to generate monthly statements

Drawbacks of using File system

- ▣ **Data redundancy and inconsistency**
 - Multiple file formats, duplication of information in different files
- ▣ **Difficulty in accessing data**
 - Need to write a new program to carry out each new task
- ▣ **Data isolation** — multiple files and formats
- ▣ **Integrity problems**
 - Integrity constraints
 - Hard to add new constraints or change existing ones

Drawbacks of using File system

▣ **Atomicity of updates**

- Failures may leave database in an inconsistent state with partial updates carried out
- E.g. transfer of funds from one account to another should either complete or not happen at all

▣ **Concurrent access by multiple users**

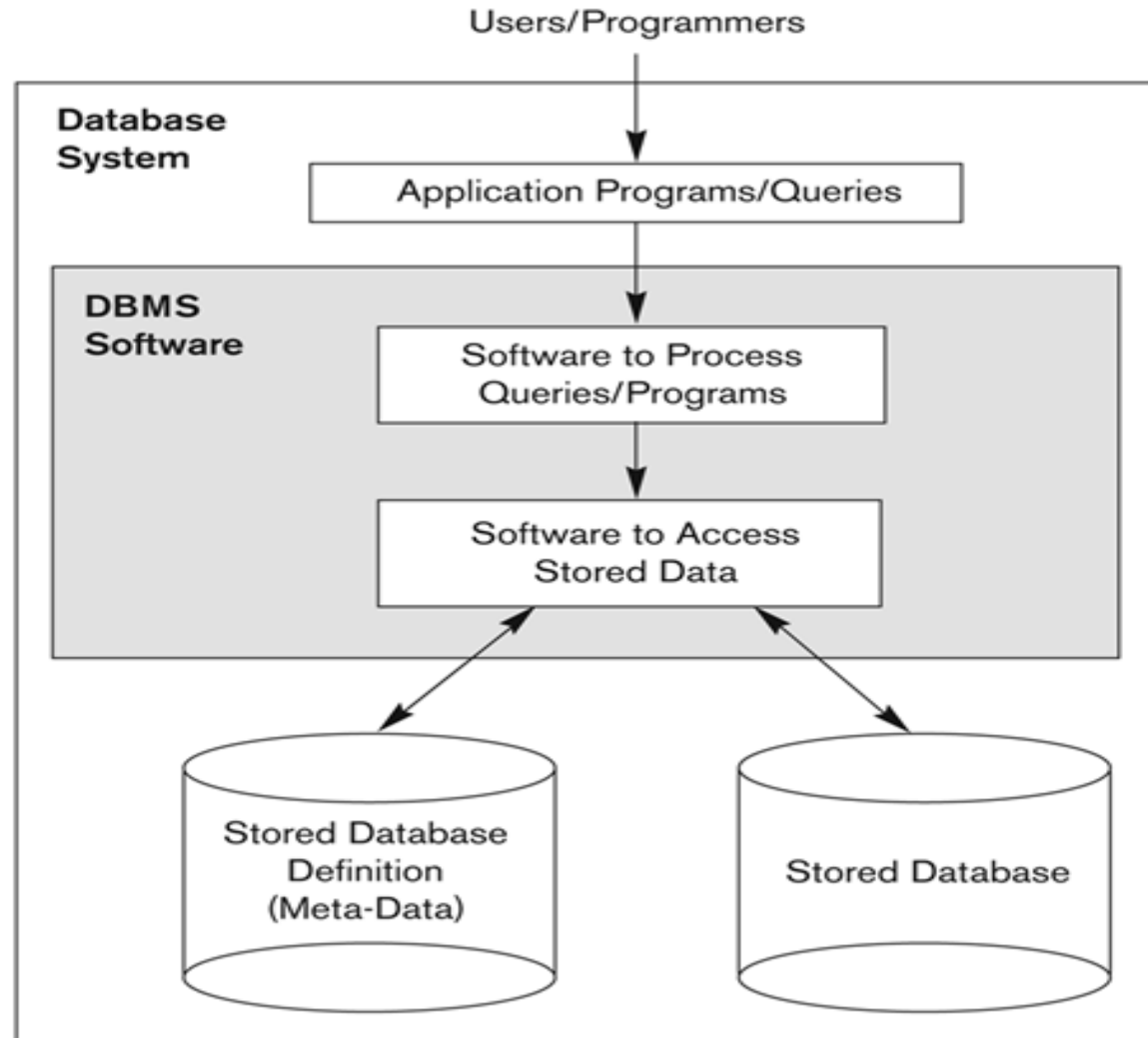
- Concurrent accessed needed for performance
- Uncontrolled concurrent accesses can lead to inconsistencies
 - E.g. two people reading a balance and updating it at the same time

▣ **Security problems**

Advantages of using a DBMS

- ❑ Controlling Redundancy
- ❑ Restricting Un-authorized Access
- ❑ Providing persistent storage for pgm objects and data structures
- ❑ Providing multiple user interfaces
- ❑ Representing complex relationships among data
- ❑ Enforcing integrity constraints
- ❑ Providing Backup and recovery

Simplified database system environment



Typical DBMS Functionality

- *Define* a particular database in terms of its data types, structures, and constraints
- *Construct* or Load the initial database contents on a secondary storage medium
- *Manipulating* the database:
 - ▣ Retrieval: Querying, generating reports
 - ▣ Modification: Insertions, deletions and updates to its content
 - ▣ Accessing the database through Web applications
- *Processing* and *Sharing* by a set of concurrent users and application programs – yet, keeping all data valid and consistent

Typical DBMS Functionality

□ Other features:

- Protection or Security measures to prevent unauthorized access
- “Active” processing to take internal actions on data
- Presentation and Visualization of data
- Maintaining the database and associated programs over the lifetime of the database application
 - Called database, software, and system maintenance

Example of a Database (with a Conceptual Data Model)

- **Mini-world for the example:**
 - ▣ Part of a UNIVERSITY environment.
- **Some mini-world *entities*:**
 - ▣ STUDENTs
 - ▣ COURSEs
 - ▣ SECTIONs (of COURSEs)
 - ▣ (academic) DEPARTMENTs
 - ▣ INSTRUCTORs

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	04	King
92	CS1310	Fall	04	Anderson
102	CS3320	Spring	05	Knuth
112	MATH2410	Fall	05	Chang
119	CS1310	Fall	05	Anderson
135	CS3380	Fall	05	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

Example
of a
simple
database

Main Characteristics of the Database Approach

- **Self-describing nature of a database system:**
 - ▣ A DBMS **catalog** stores the description of a particular database (e.g. data structures, types, and constraints)
 - ▣ The description is called **meta-data**.
 - ▣ This allows the DBMS software to work with different database applications.
- **Insulation between programs and data:**
 - ▣ Called **program-data independence**.
 - ▣ Allows changing data structures and storage organization without having to change the DBMS access programs.

Example of a simplified database catalog

RELATIONS

Relation_name	No_of_columns
STUDENT	4
COURSE	4
SECTION	5
GRADE_REPORT	3
PREREQUISITE	2

COLUMNS

Column_name	Data_type	Belongs_to_relation
Name	Character (30)	STUDENT
Student_number	Character (4)	STUDENT
Class	Integer (1)	STUDENT
Major	Major_type	STUDENT
Course_name	Character (10)	COURSE
Course_number	XXXXNNNN	COURSE
....		
....		
....		
Prerequisite_number	XXXXNNNN	PREREQUISITE

Main Characteristics of the Database Approach ¹⁶

(continued)

□ **Data Abstraction:**

- ▣ A **data model** is used to hide storage details and present the users with a conceptual view of the database.
- ▣ Programs refer to the data model constructs rather than data storage details

□ **Support of multiple views of the data:**

- ▣ Each user may see a different view of the database, which describes **only** the data of interest to that user.

Main Characteristics of the Database Approach(continued)

- **Sharing of data and multi-user transaction processing:**
 - ▣ Allowing a set of **concurrent users** to retrieve from and to update the database.
 - ▣ *Concurrency control* within the DBMS guarantees that each **transaction** is correctly executed or aborted
 - ▣ *Recovery* subsystem ensures each completed transaction has its effect permanently recorded in the database
 - ▣ **OLTP** (Online Transaction Processing) is a major part of database applications. This allows hundreds of concurrent transactions to execute per second.

Additional Implications of Using the Database Approach

- Potential for enforcing standards:
 - ▣ This is very crucial for the success of database applications in large organizations. **Standards** refer to data item names, display formats, screens, report structures, meta-data (description of data), Web page layouts, etc.
- Reduced application development time:
 - ▣ Incremental time to add each new application is reduced.

Additional Implications of Using the Database Approach (continued)

- Flexibility to change data structures:
 - ▣ Database structure may evolve as new requirements are defined.
- Availability of current information:
 - ▣ Extremely important for on-line transaction systems such as airline, hotel, car reservations.
- Economies of scale:
 - ▣ Wasteful overlap of resources and personnel can be avoided by consolidating data and applications across departments.

When not to use a DBMS

- Main inhibitors (costs) of using a DBMS:
 - ▣ High initial investment and possible need for additional hardware.
 - ▣ Overhead for providing generality, security, concurrency control, recovery, and integrity functions.
- When a DBMS may be unnecessary:
 - ▣ If the database and applications are simple, well defined, and not expected to change.
 - ▣ If there are stringent real-time requirements that may not be met because of DBMS overhead.
 - ▣ If access to data by multiple users is not required.

When not to use a DBMS

- When no DBMS may suffice:
 - ▣ If the database system is not able to handle the complexity of data because of modeling limitations
 - ▣ If the database users need special operations not supported by the DBMS.

Data Models

□ Data Model:

- A set of concepts to describe the **structure** of a database, the **operations** for manipulating these structures, and certain **constraints** that the database should obey.

□ Data Model Structure and Constraints:

- Constructs are used to define the database structure
- Constructs typically include **elements** (and their **data types**) as well as groups of elements (e.g. **entity**, **record**, **table**), and **relationships** among such groups
- Constraints specify some restrictions on valid data; these constraints must be enforced at all times

□ Data Model Operations:

- These operations are used for specifying database *retrievals* and *updates* by referring to the constructs of the data model.
- Operations on the data model may include ***basic model operations*** (e.g. generic insert, delete, update) and ***user-defined operations*** (e.g. compute_student_gpa, update_inventory)

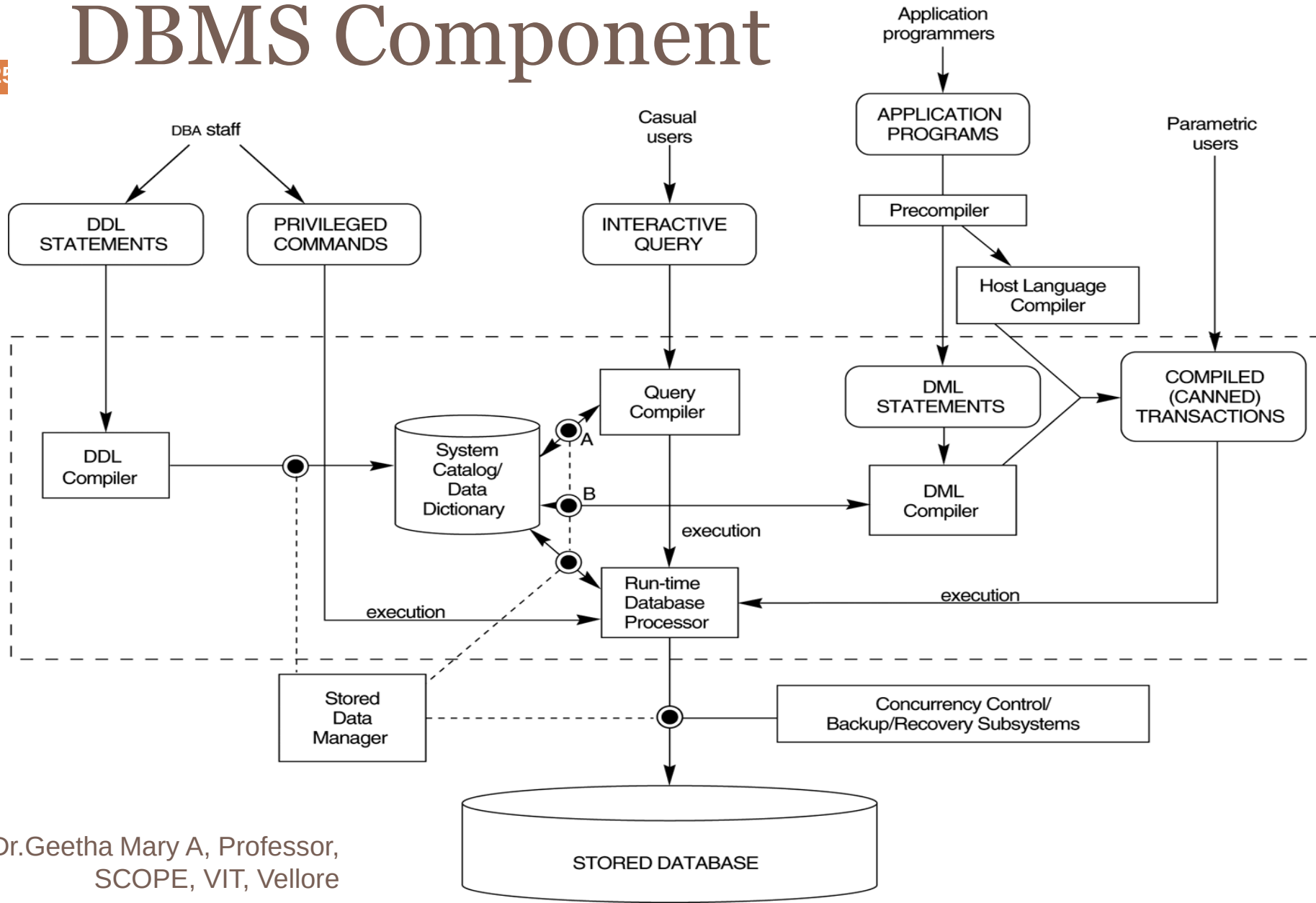
Categories of Data Models

- **Conceptual (high-level, semantic) data models:**
 - ▣ Provide concepts that are close to the **way many users perceive data**.
 - (Also called *entity-based* or *object-based* data models.)
- **Physical (low-level, internal) data models:**
 - ▣ Provide concepts that describe details of **how data is stored in the computer**. These are usually specified in an ad-hoc manner through DBMS design and administration manuals
- **Implementation (representational) data models:**
 - ▣ Provide concepts that fall between the above two, used by many commercial DBMS implementations (e.g. relational data models used in many commercial systems).

Database System Environment

DBMS Component

25



Database Users

- Users may be divided into
 - ▣ Those who actually use and control the database content, and those who design, develop and maintain database applications (called “**Actors on the Scene**”), and
 - ▣ Those who design and develop the DBMS software and related tools, and the computer systems operators (called “**Workers Behind the Scene**”).

Database Users

□ Actors on the scene

▣ **Database administrators:**

- Responsible for authorizing access to the database, for coordinating and monitoring its use, acquiring software and hardware resources, controlling its use and monitoring efficiency of operations.

▣ **Database Designers:**

- Responsible to define the content, the structure, the constraints, and functions or transactions against the database. They must communicate with the end-users and understand their needs.

Categories of End-users

- Actors on the scene (continued)
 - ▣ **End-users:** They use the data for queries, reports and some of them update the database content. End-users can be categorized into:
 - **Casual:** access database occasionally when needed
 - **Naïve** or Parametric: they make up a large section of the end-user population.
 - They use previously well-defined functions in the form of “canned transactions” against the database.
 - Examples are bank-tellers or reservation clerks who do this activity for an entire shift of operations.

Categories of End-users (continued)

■ **Sophisticated:**

- These include business analysts, scientists, engineers, others thoroughly familiar with the system capabilities.
- Many use tools in the form of software packages that work closely with the stored database.

■ **Stand-alone:**

- Mostly maintain personal databases using ready-to-use packaged applications.
- An example is a tax program user that creates its own internal database.
- Another example is a user that maintains an address book

DBMS Functions

- ❑ Data Dictionary Management
- ❑ Data Storage Management
- ❑ Data Transformation and Presentation
- ❑ Security Management
- ❑ Multiuser Access Control
- ❑ Backup and Recovery Management
- ❑ Data Integrity Management
- ❑ Database Access Languages and Application Programming Interfaces
- ❑ Database Communication Interfaces
- ❑ Transaction Management

Schemas versus Instances

- Database Schema:
 - ▣ The ***description*** of a database.
 - ▣ Includes descriptions of the database structure, data types, and the constraints on the database.
- Schema Diagram:
 - ▣ An ***illustrative*** display of (most aspects of) a database schema.
- Schema Construct:
 - ▣ A ***component*** of the schema or an object within the schema, e.g., STUDENT, COURSE.

Schemas versus Instances

□ Database State:

- ▣ The actual data stored in a database at a ***particular moment in time***. This includes the collection of all the data in the database.
- ▣ Also called database instance (or occurrence or snapshot).
 - The term *instance* is also applied to individual database components, e.g. *record instance*, *table instance*, *entity instance*

Database Schema vs. Database State

- Database State:
 - ▣ Refers to the ***content*** of a database at a moment in time.
- Initial Database State:
 - ▣ Refers to the database state when it is initially loaded into the system.
- Valid State:
 - ▣ A state that satisfies the structure and constraints of the database.

Database Schema vs. Database State (continued)

- Distinction
 - ▣ The ***database schema*** changes very infrequently.
 - ▣ The ***database state*** changes every time the database is updated.
- **Schema** is also called **intension**.
- **State** is also called **extension**.

Example of a Database Schema

STUDENT

Name	Student_number	Class	Major
------	----------------	-------	-------

COURSE

Course_name	Course_number	Credit_hours	Department
-------------	---------------	--------------	------------

PREREQUISITE

Course_number	Prerequisite_number
---------------	---------------------

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
--------------------	---------------	----------	------	------------

GRADE_REPORT

Student_number	Section_identifier	Grade
----------------	--------------------	-------

Example of a database state

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	04	King
92	CS1310	Fall	04	Anderson
102	CS3320	Spring	05	Knuth
112	MATH2410	Fall	05	Chang
119	CS1310	Fall	05	Anderson
135	CS3380	Fall	05	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

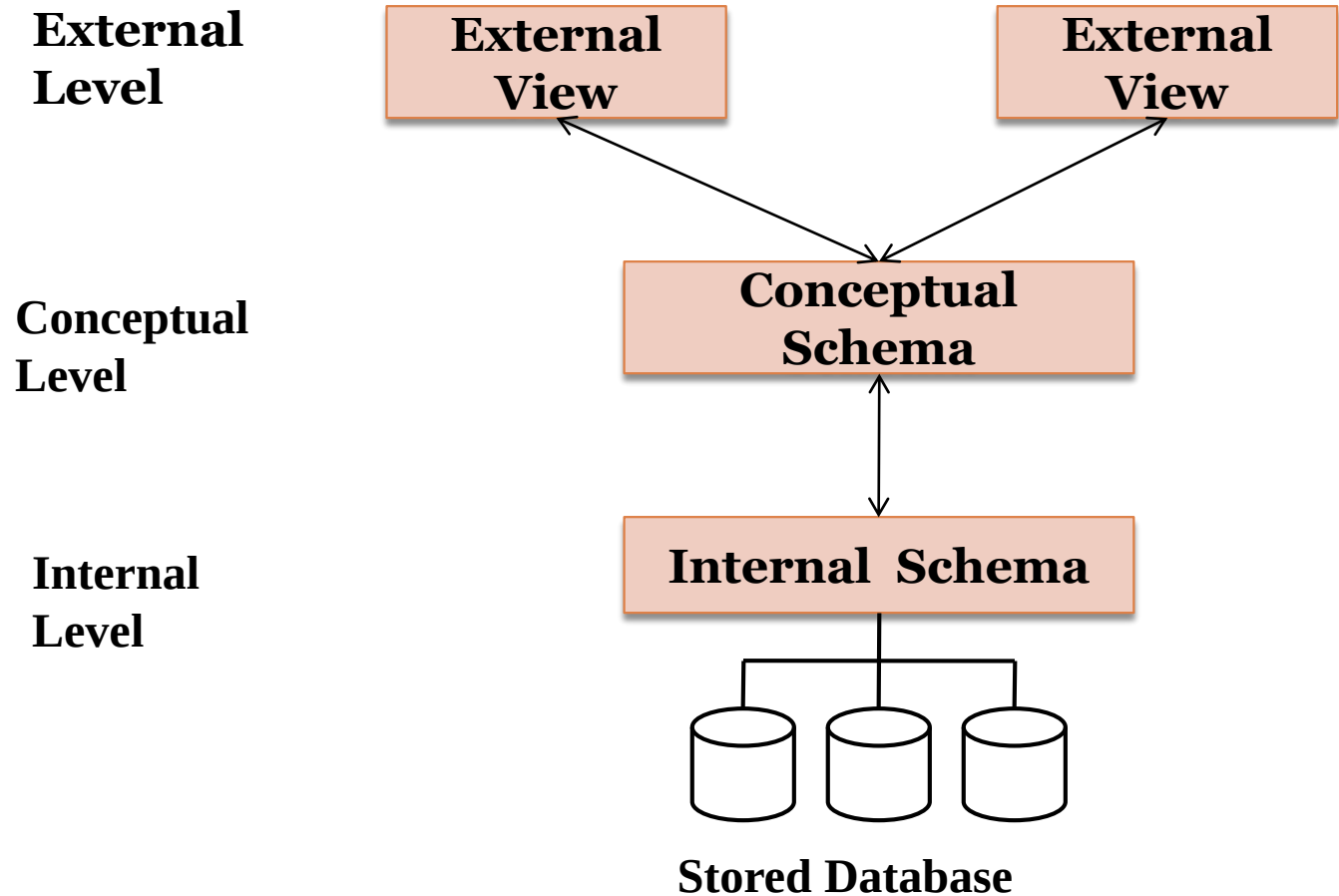
Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

Three schema architecture

- Important characteristics of database approach:
 - Insulation of programs and data
 - Support multiple user views
 - Use of a catalog to store the database description
(Schema)
- An architecture of schema is proposed to achieve these characteristics
- Architecture goal: To separate the user application and the database

Three schema architecture

38



Three-Schema Architecture

Defines DBMS schemas at *three levels*:

- **Internal schema** at the internal level to describe physical storage structures and access paths. Typically uses a *physical* data model.
- **Conceptual schema** at the conceptual level to describe the structure and constraints for the *whole* database for a community of users. Uses a *conceptual* or an *implementation* data model.
- **External schemas** at the external level to describe the various user views. Usually uses the same data model as the conceptual level.

Three-Schema Architecture



Mappings among schema levels are needed to transform requests and data. Programs refer to an external schema, and are mapped by the DBMS to the internal schema for execution.

Access path

Structure that makes the search for particular database records efficient

```
select * from employee where sal>1000 and empid>20
```

empid	Name	Sal	place
5	Pavithra	5000	Chennai
2	Kirthika	10000	Bangalore
55	Amrith	15000	Chennai
25	Jai	20000	Bangalore
33	Anu	26000	Chennai

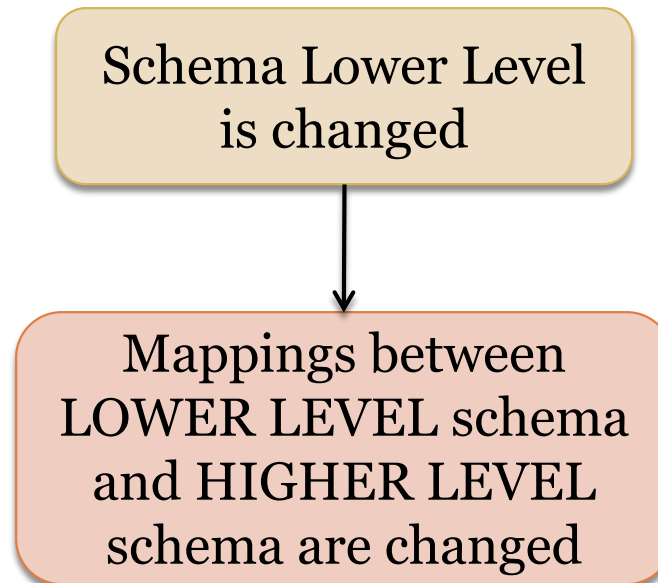
- Which condition need to be applied first.
- Query optimizing

Data Independence

Capacity to change the schema at one level of database system without having to change the schema at the next higher level

- **Logical Data Independence:** The capacity to change the conceptual schema without having to change the external schemas and their application programs.
- **Physical Data Independence:** The capacity to change the internal schema without having to change the conceptual schema.

Data Independence



Higher level schemas are unchanged.

DBMS Languages

- Data Definition Language (DDL)
- Data Manipulation Language (DML)
 - ▣ High-Level or Non-procedural Languages: These include the relational language SQL
 - May be used in a standalone way or may be embedded in a programming language
 - ▣ Low Level or Procedural Languages:
 - These must be embedded in a programming language

DBMS Languages

- **Data Definition Language (DDL):**
 - ▣ Used by the DBA and database designers to specify the conceptual schema of a database.
 - ▣ In many DBMSs, the DDL is also used to define internal and external schemas (views).
 - ▣ In some DBMSs, separate **storage definition language (SDL)** and **view definition language (VDL)** are used to define internal and external schemas.
 - SDL is typically realized via DBMS commands provided to the DBA and database designers

DBMS Languages

- **Data Manipulation Language (DML):**
 - ▣ Used to specify database retrievals and updates
 - ▣ DML commands (data sublanguage) can be *embedded* in a general-purpose programming language (host language), such as COBOL, C, C++, or Java.
 - A library of functions can also be provided to access the DBMS from a programming language
 - ▣ Alternatively, stand-alone DML commands can be applied directly (called a *query language*).

Types of DML

□ High Level or Non-procedural Language:

- ▣ For example, the SQL relational language
- ▣ Are “set”-oriented and specify what data to retrieve rather than how to retrieve it.
- ▣ Also called **declarative** languages.

□ Low Level or Procedural Language:

- ▣ Retrieve data one record-at-a-time;
- ▣ Constructs such as looping are needed to retrieve multiple records, along with positioning pointers.

DBMS Interfaces

- Stand-alone query language interfaces
 - ▣ Example: Entering SQL queries at the DBMS interactive SQL interface (e.g. SQL*Plus in ORACLE)
- Programmer interfaces for embedding DML in programming languages
- User-friendly interfaces
 - ▣ Menu-based, forms-based, graphics-based, etc.

DBMS Programming Language Interfaces

- Programmer interfaces for embedding DML in a programming languages:
 - ▣ **Embedded Approach:** e.g. embedded SQL (for C, C++, etc.), SQLJ (for Java)
 - ▣ **Procedure Call Approach:** e.g. JDBC for Java, ODBC for other programming languages
 - ▣ **Database Programming Language Approach:** e.g. ORACLE has PL/SQL, a programming language based on SQL; language incorporates SQL and its data types as integral components

User-Friendly DBMS Interfaces

- ▣ Menu-based, popular for browsing on the web
- ▣ Forms-based, designed for naïve users
- ▣ Graphics-based
 - (Point and Click, Drag and Drop, etc.)
- ▣ Natural language: requests in written English
- ▣ Combinations of the above:
 - For example, both menus and forms used extensively in Web database interfaces

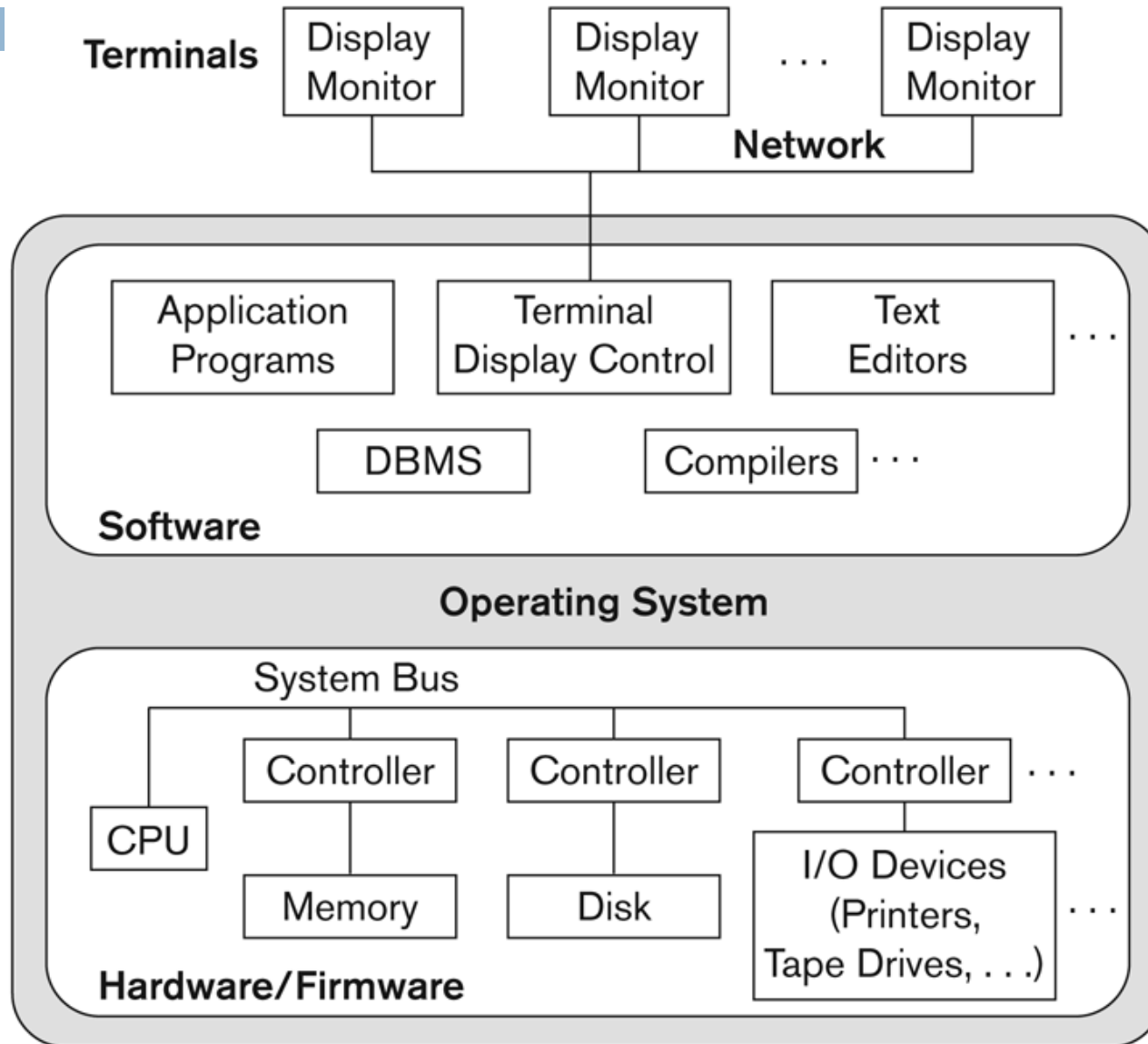
Other DBMS Interfaces

- ▣ Speech as Input and Output
- ▣ Web Browser as an interface
- ▣ Parametric interfaces, e.g., bank tellers using function keys.
- ▣ Interfaces for the DBA:
 - Creating user accounts, granting authorizations
 - Setting system parameters
 - Changing schemas or access paths

Centralized and Client-Server DBMS Architectures

- Centralized DBMS:
 - ▣ Combines everything into single system including- DBMS software, hardware, application programs, and user interface processing software.
 - ▣ User can still connect through a remote terminal – however, all processing is done at centralized site.

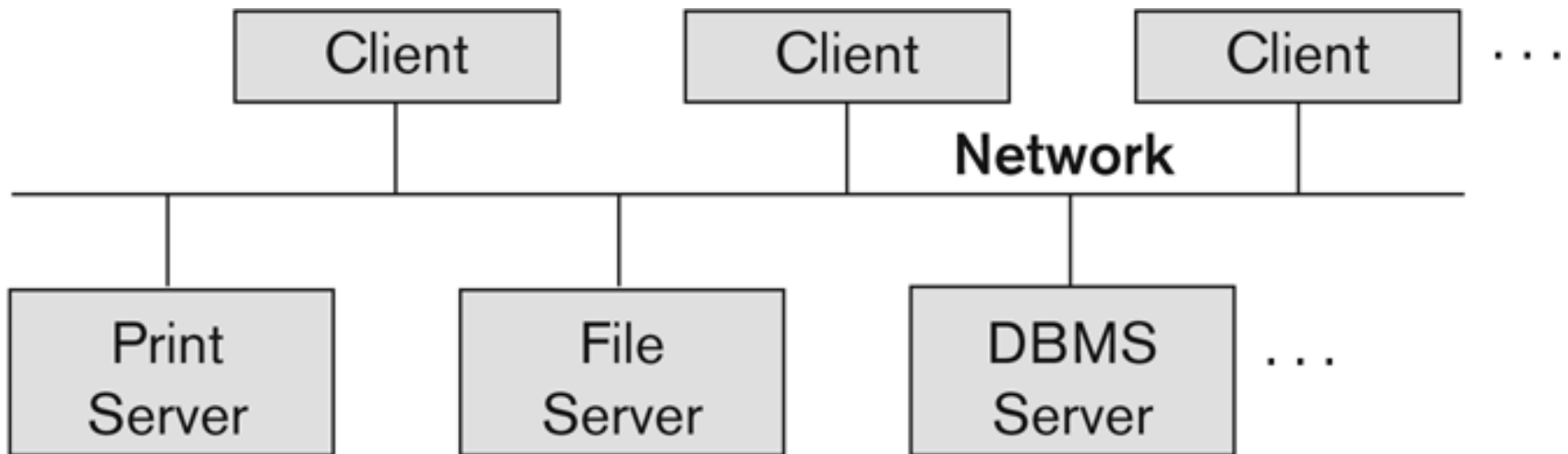
A Physical Centralized Architecture



Basic 2-tier Client-Server Architectures

- Specialized Servers with Specialized functions
 - ▣ Print server
 - ▣ File server
 - ▣ DBMS server
 - ▣ Web server
 - ▣ Email server
- Clients can access the specialized servers as needed

Logical two-tier client server architecture



Clients

- ❑ Provide appropriate interfaces through a client software module to access and utilize the various server resources.
- ❑ Clients may be diskless machines or PCs or Workstations with disks with only the client software installed.
- ❑ Connected to the servers via some form of a network.
 - ❑ (LAN: local area network, wireless network, etc.)

DBMS Server

- Provides database query and transaction services to the clients
- Relational DBMS servers are often called SQL servers, query servers, or transaction servers
- Applications running on clients utilize an Application Program Interface (**API**) to access server databases via standard interface such as:
 - ▣ ODBC: Open Database Connectivity standard
 - ▣ JDBC: for Java programming access
- Client and server must install appropriate client module and server module software for ODBC or JDBC

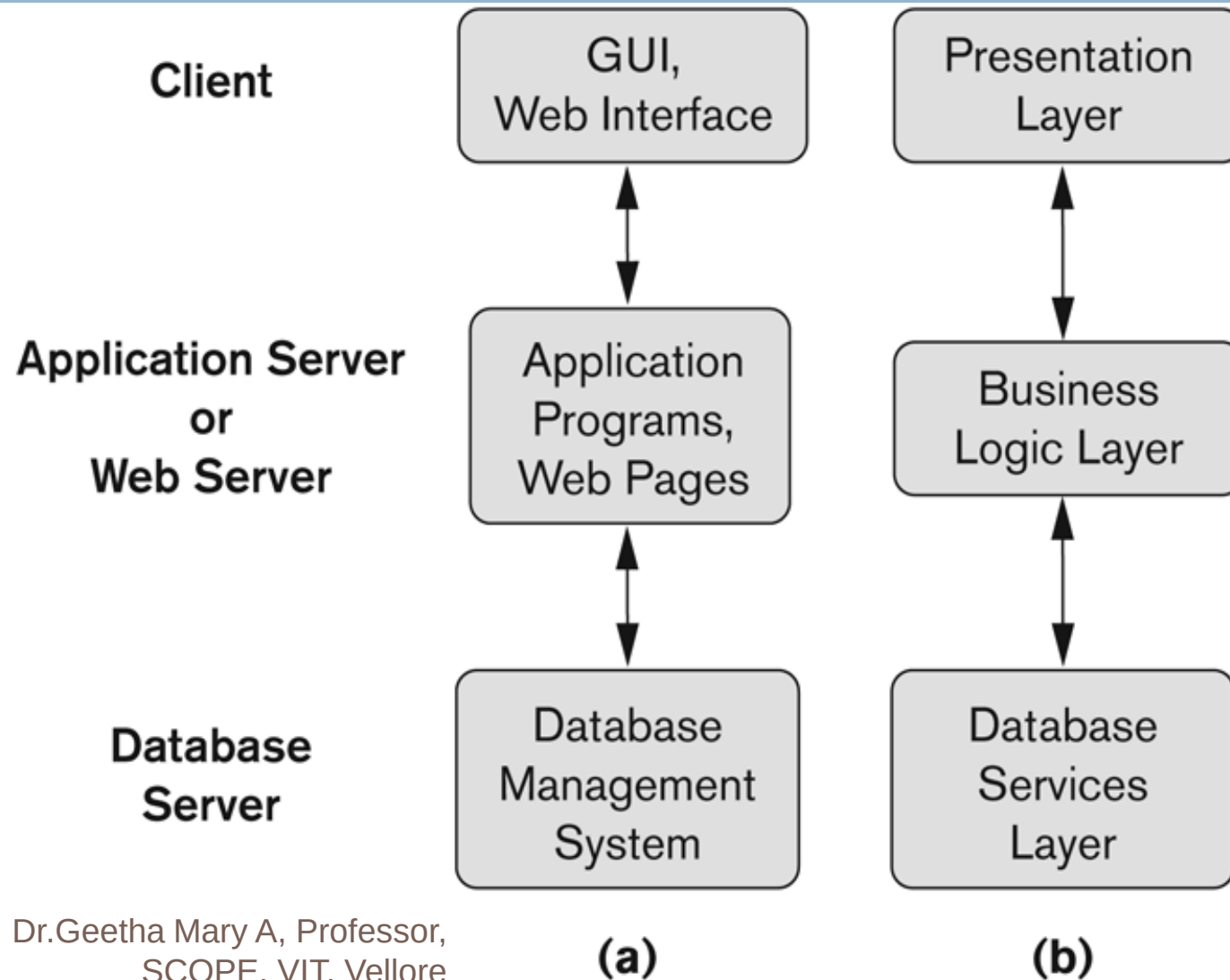
Two Tier Client-Server Architecture

- A client program may connect to several DBMSs, sometimes called the data sources.
- In general, data sources can be files or other non-DBMS software that manages data.
- Other variations of clients are possible: e.g., in some object DBMSs, more functionality is transferred to clients including data dictionary functions, optimization and recovery across multiple servers, etc.

Three Tier Client-Server Architecture

- ❑ Common for Web applications
- ❑ Intermediate Layer called Application Server or Web Server:
 - ▣ Stores the web connectivity software and the business logic part of the application used to access the corresponding data from the database server
 - ▣ Acts like a conduit for sending partially processed data between the database server and the client.
- ❑ Three-tier Architecture Can Enhance Security:
 - ▣ Database server only accessible via middle tier
 - ▣ Clients cannot directly access database server

Three-tier client-server architecture

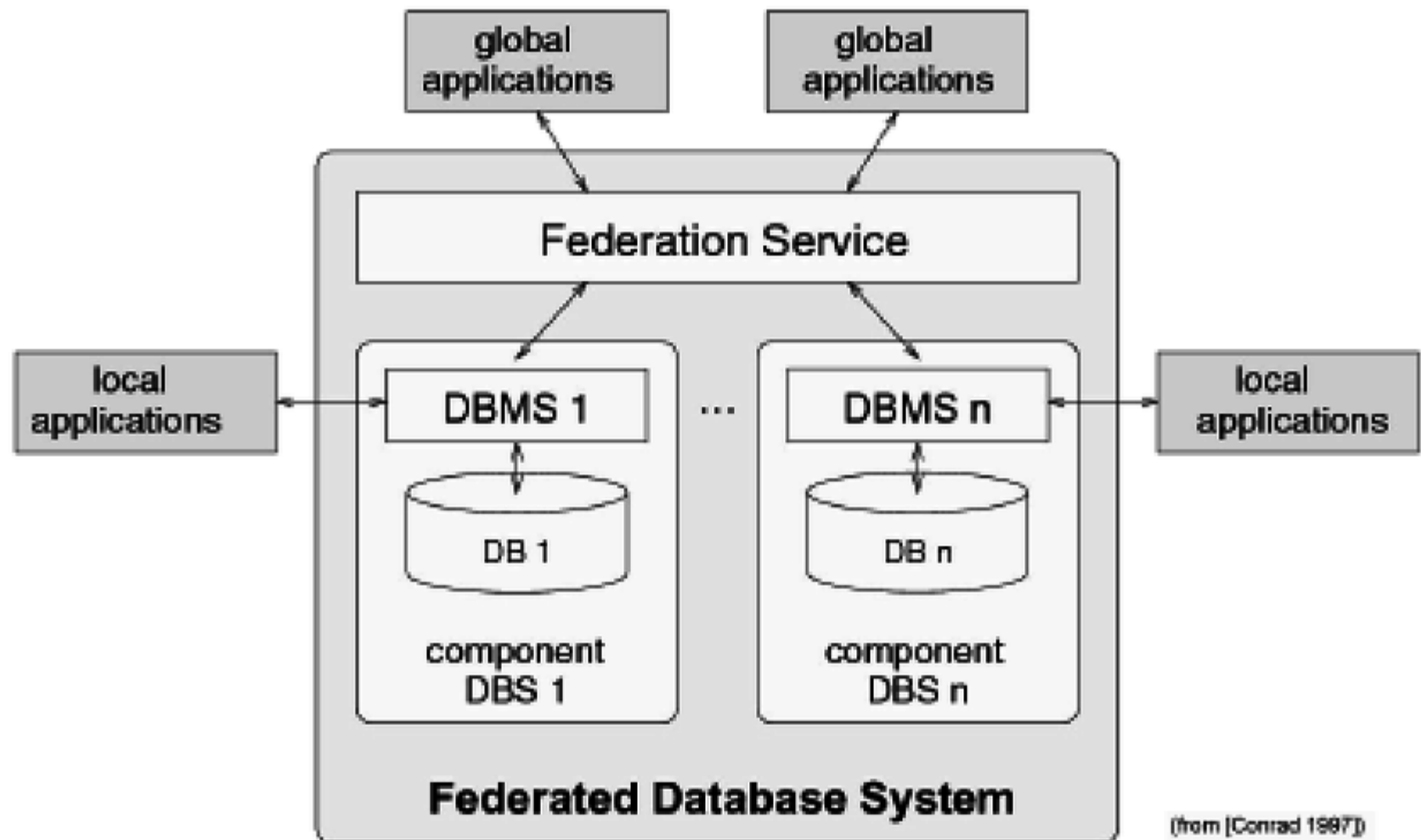


Classification of DBMSs

- Based on the data model used
 - ▣ Traditional: Relational, Network, Hierarchical.
 - ▣ Emerging: Object-oriented, Object-relational.
- Other classifications
 - ▣ Single-user (typically used with personal computers)
vs. multi-user (most DBMSs).
 - ▣ Centralized (uses a single computer with one database)
vs. distributed (uses multiple computers, multiple databases)

Variations of Distributed DBMSs (DDBMSs)

- Homogeneous DDBMS
- Heterogeneous DDBMS
- Federated or Multidatabase Systems
- Distributed Database Systems have now come to be known as client-server based database systems because:
 - ▣ They do not support a totally distributed environment, but rather a set of database servers supporting a set of clients.

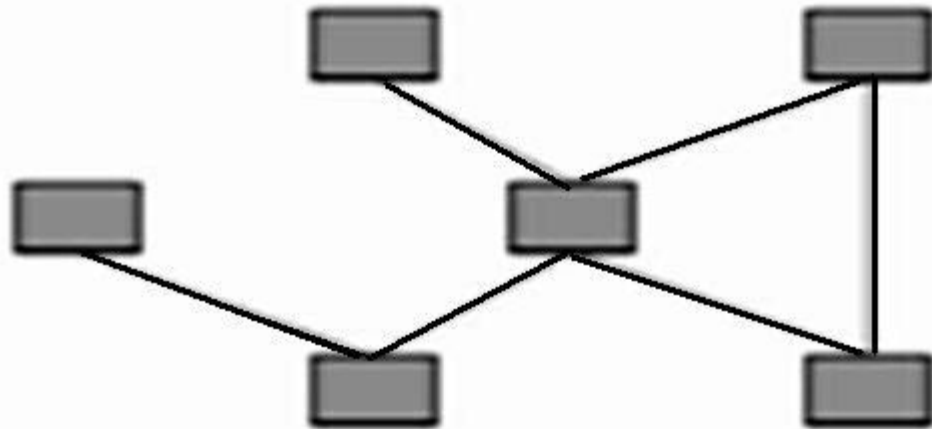


History of Data Models

- ❑ Network Model
- ❑ Hierarchical Model
- ❑ Relational Model
- ❑ Object Oriented Model
- ❑ Object Relational Model

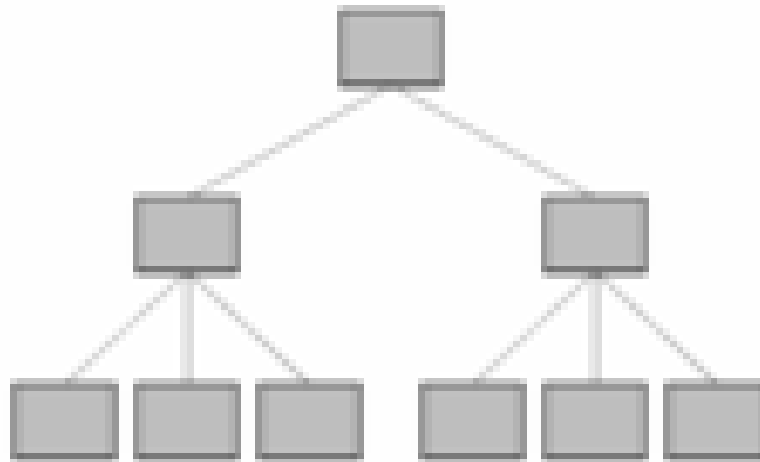
Data Model - Network model

- Organizes data using two fundamental constructs, called records and sets. Records contain fields, and sets define one-to-many relationships between records: one owner, many members.



Data Model - Hierarchical model

- Data is organized into a tree-like structure → single upward link in each record to describe the nesting, and a sort field to keep the records in a particular order in each same-level list.



Data Model - Relational model

- Based on first-order predicate logic. Its core idea is to describe a database as a collection of predicates over a finite set of predicate variables, describing constraints on the possible values and combinations of values.

Relational Model

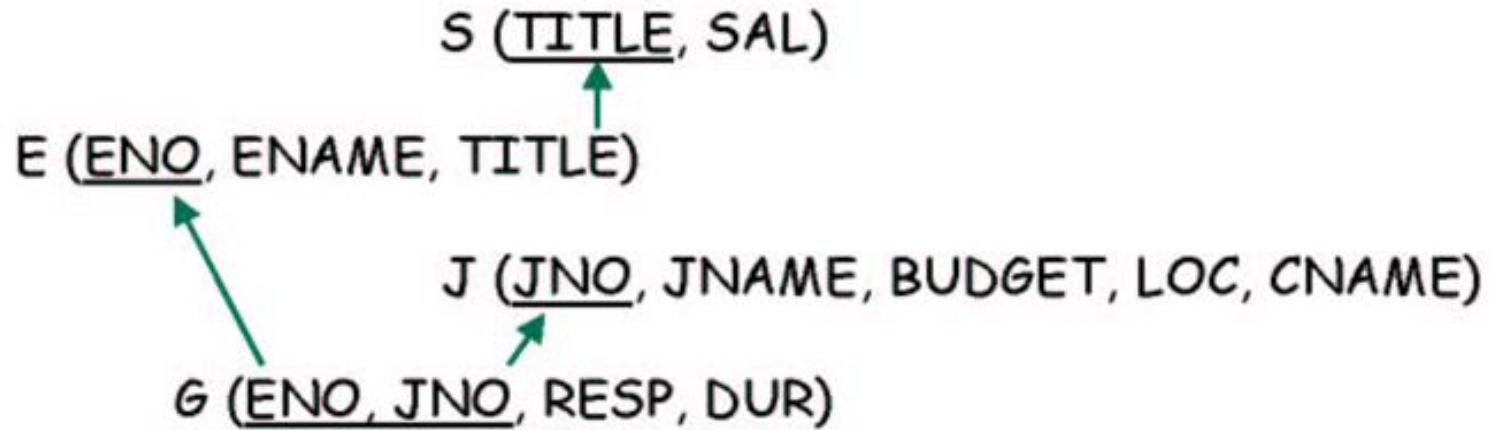
Activity Code	Activity Name
23	Patching
24	Overlay
25	Crack Sealing

Key = 24

Activity Code	Date	Route No.
24	01/12/01	I-95
24	02/08/01	I-66

Date	Activity Code	Route No.
01/12/01	24	I-95
01/15/01	23	I-495
02/08/01	24	I-66

Relational Model



Network Model

